XV6 RACE CONDITIONS

Four-Member Work Division and Deep-Dive Notes

Case Study 14: Race Condition in XV6 Shared Data. A balanced division with a clear difficulty ranking.

All members should first read the **Common Foundation Guide**. This document assigns ownership of the deeper topics. Every member gets an equal 25% share of the total work. **Member 1 (you) takes the hardest task.**

Equal workload

Each member produces roughly the same amount: about **2 report pages**, one **3 to 4 minute presentation segment**, one diagram or table, and about **6 to 8 hours** of effort. Harder tasks involve less writing; easier tasks include extra writing and compiling, which keeps the total load equal.

Member	Primary ownership	Share	Difficulty rank
Member 1 (You / technical lead)	Hands-on experiment: remove lock, run concurrent test, restore lock, compare	25%	**1st, MOST difficult**
Member 2	Lock-removal analysis: race scenarios and interleaving timelines	25%	2nd (hard)
Member 3	Locking mechanism: spinlock internals and critical sections	25%	3rd (moderate)
Member 4	Foundations, shared-data inventory, report compilation and evidence audit	25%	**4th, LEAST difficult**

Why Member 1 is the most difficult

It needs a working XV6 build under QEMU, editing and rebuilding the kernel, writing a multi-process stress program, and diagnosing failures that are **timing-dependent and may not appear on every run**. Kernel panics and hangs must be interpreted, and the lock-free and locked runs must be compared fairly.

Why Member 4 is the least difficult

It is mostly theory and source browsing plus compiling the report. It carries the extra formatting and auditing work so the total effort stays equal.

Shared working rules

- Use one repository revision and one tested program across the group.
- Keep a shared folder with source code, exact commands, screenshots, source notes and the results table.
- Each lead teaches their section to the other three; everyone reviews the diagram and the observed results.
- Any claim that cannot be explained or verified in source or experiment must be corrected.
- Do not fabricate screenshots, outputs, panics, crash counts or contributions. If the race never appears, report that honestly.
- Always restore `kernel/kalloc.c` to its original locked form before final screenshots and submission.

MEMBER 1 — You: hands-on experiment (hardest section)

Mission

Demonstrate the race for real. Remove the lock from the free-list code, run a concurrent stress test on multiple CPUs, record what goes wrong, restore the lock, and compare. This supports the objective *observe the effect of concurrent updates* and the activities on removing the lock, recording incorrect outcomes and restoring locking.

What you must understand

- Which XV6 operations reach `kalloc()` and `kfree()` (for example `sbrk()` growing/shrinking memory, `fork()`, `exec()`).
- How to run XV6 with several CPUs: `make qemu CPUS=4`. Confirm the syntax in your Makefile.
- Why a race may not show on every run, and how to increase the chance (more CPUs, more children, more iterations).
- How to detect a double-allocated page from user space: each child fills its pages with a unique value and later checks them.
- How to compare lock vs no-lock fairly: same program, same CPU count, same number of runs.

Step-by-step plan

Step	Action
1	Record the revision. Build and boot XV6: `make qemu CPUS=4`. Run `usertests` or a quick sanity check once to confirm a healthy baseline.
2	Add the stress program (below) to UPROGS in the Makefile. Rebuild.
3	**Baseline:** run the program 10+ times with locking intact. Record each run.
4	Back up `kernel/kalloc.c` (git or copy). Comment out `acquire(&kmem.lock)` and `release(&kmem.lock)` in **both** `kalloc()` and `kfree()`. Screenshot the `git diff`.
5	Rebuild and run the same program 10+ times with the same CPU count. Record panics, hangs, corruption messages, or clean passes.
6	*Optional, label clearly as instrumented:* add a short busy loop between reading `kmem.freelist` and updating it, to widen the race window. Report it separately from the unmodified lock-free run.
7	Restore locking (`git checkout kernel/kalloc.c`), rebuild, rerun the same number of times. Compare.
8	Fill the results table and write the conclusion.

Suggested stress program (skeleton; verify against your revision)

Idea: several children repeatedly grow memory with `sbrk()`, write a unique tag into every page, spin briefly, check the tag, then shrink memory. If two children receive the same physical page, a tag mismatch appears. Check how `printf` and `sbrk` behave in your version (`printf` has no fd argument in current xv6-riscv).

```
// user/race_test.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define NCHILD 4
#define NPAGES 8
#define ROUNDS 200
#define PG 4096

int child(int id) {
  char tag = 'A' + id;
  for(int r = 0; r < ROUNDS; r++){
    char *base = sbrk(NPAGES * PG);
    if(base == (char*)-1){ printf("child %d: sbrk failed\n", id); return 1; }
    for(int p = 0; p < NPAGES; p++)
      for(int i = 0; i < PG; i += 64)
        base[p*PG + i] = tag;
    for(volatile int d = 0; d < 2000; d++) ;        // give others time to interfere
```

```
    for(int p = 0; p < NPAGES; p++)
      for(int i = 0; i < PG; i += 64)
        if(base[p*PG + i] != tag){
          printf("child %d: CORRUPTION round %d page %d\n", id, r, p);
          return 1;
        }
    sbrk(-NPAGES * PG);
  }
  return 0;
}

int main(void) {
  int bad = 0, st;
  for(int c = 0; c < NCHILD; c++){
    int pid = fork();
    if(pid < 0){ printf("fork failed\n"); exit(1); }
    if(pid == 0) exit(child(c));
  }
  for(int c = 0; c < NCHILD; c++){ wait(&st); if(st != 0) bad++; }
  printf("race_test: %d of %d children reported problems\n", bad, NCHILD);
  exit(bad ? 1 : 0);
}
```

Add `$U/_race_test\` to `UPROGS` following the existing lines. Tune `NCHILD`, `NPAGES` and `ROUNDS` together with the CPU count.

Results table template

Run	CPUs	Lock	Expected	Observed (real result only)
1 to 10	4	Present	No corruption, no panic	[fill with real result]
11 to 20	4	Removed	Possible corruption, panic, hang or clean pass	[fill with real result]
21 to 30	4	Restored	No corruption, no panic	[fill with real result]

Your deliverables

- Screenshots: `git diff` of the lock removal, lock-free runs, locked runs.
- Completed before vs after results table covering every run.
- An honest conclusion: what was observed, how often, and what remains untested.
- A 60 to 90 second explanation of the experiment.

Oral explanation scaffold

> *We removed the lock from kalloc and kfree and ran several processes that allocate and free memory on multiple CPUs. Without the lock, we observed [real result]. After restoring the lock and repeating the same test, we observed [real result]. This shows that the lock protects the free list from concurrent updates. Because the race depends on timing, passing runs alone would not prove the code is safe.*

Questions to prepare for

- Why did the failure not appear on every lock-free run?
- Why does a clean lock-free run not prove the code is correct?
- Why does a double-allocated page cause corruption?
- Why must `kfree()` be locked as well as `kalloc()`?
- Why use more than one CPU, and could a race happen on one CPU?
- What would a deadlock look like compared to what you saw?

MEMBER 2 — Lock-removal analysis and race scenarios

Mission

Explain, on paper, exactly what can go wrong when the lock is removed. Produce clear step-by-step interleavings of two CPUs and list every incorrect outcome. This supports the activities on analysing lock removal, constructing a two-CPU example and recording possible incorrect outcomes.

Topics to learn

- Read-modify-write on the free list in `kalloc()` and `kfree()`; why each is a critical section.
- How interleavings are written: one row per step, one column per CPU, plus the data state after each step.
- A second structure for comparison: the scheduler in `kernel/proc.c`, where `p->lock` stops two CPUs from picking the same RUNNABLE process.
- Possible incorrect outcomes: double allocation, lost (leaked) pages, corrupted list pointers, a process running on two CPUs, duplicate PIDs.
- How the lock removes each incorrect outcome by forcing the critical sections to run one at a time.

Commands

```
cd ~/xv6-riscv
sed -n '1,100p' kernel/kalloc.c         # read kfree and kalloc fully
grep -n 'scheduler\|RUNNABLE\|allocproc\|allocpid' kernel/proc.c
sed -n 'START,ENDp' kernel/proc.c       # read scheduler() and allocproc() in full
```

Run these in Ubuntu/WSL at the repository root, not inside the XV6 emulator shell. Read the surrounding functions; search results only help you navigate.

Scheduler race for your second scenario

Step	CPU 0	CPU 1	Without `p->lock`
1	sees p->state == RUNNABLE		
2		sees p->state == RUNNABLE	both saw the same process
3	sets RUNNING, switches to p	sets RUNNING, switches to p	**p runs on two CPUs**, sharing one kernel stack
With lock	acquires `p->lock`, sets RUNNING	blocks on `p->lock`, then sees RUNNING and skips	only one CPU runs p

Your deliverables

- Interleaving timeline for the `kalloc()` race and for the `kfree()` race (use the Foundation Guide tables as a starting point, then verify against source).
- A timeline for at least one other structure (scheduler or PID allocation).
- A table: incorrect outcome, cause, which line or step causes it, how the lock prevents it.
- A review of Member 1's results so the observed behaviour matches the theoretical scenarios (or explains differences).

Oral explanation scaffold

> *Without the lock, two CPUs can both read the head of the free list before either updates it. Both receive the same page, so the page is allocated twice. In kfree, both CPUs link to the same old head and the second update overwrites the first, so a freed page is lost. With the lock, the second CPU waits, so each read-modify-write completes before the next begins.*

Questions to prepare for

- Is every interleaving harmful? Which ones happen to give correct results?
- What happens if a kalloc and a kfree interleave?
- Why does the scheduler need a lock per process rather than only one global lock?
- Why do these bugs appear rarely and unpredictably?

MEMBER 3 — Locking mechanism and critical sections

Mission

Explain how XV6 actually implements mutual exclusion and identify the exact critical sections and locks in the source. This supports the objectives on identifying critical sections and understanding how locks prevent inconsistent results.

Topics to learn

- `struct spinlock` in `kernel/spinlock.h`: fields `locked`, `name`, `cpu`.
- `acquire()` and `release()` in `kernel/spinlock.c`: atomic swap, memory fence, `holding()` check and the panics they trigger.
- `push_off()` and `pop_off()`: nested interrupt disabling and why it matters.
- Why a spinlock is correct only if every access to the shared data takes the same lock.
- Spinlock vs sleep-lock (`kernel/sleeplock.c`) and when each is used.
- Lock discipline: do not hold a spinlock across a sleep; take locks in a consistent order to avoid deadlock.

Source navigation

```
cd ~/xv6-riscv
cat kernel/spinlock.h
grep -n 'acquire\|release\|holding\|push_off\|pop_off' kernel/spinlock.c
grep -n 'kmem\|acquire\|release' kernel/kalloc.c
grep -n 'initlock' kernel/*.c          # lists where each lock is created
```

Use `grep initlock` to build the complete lock list, then confirm each protected structure by reading its file.

Your deliverables

- Flow diagram: acquire -> critical section -> release, showing where interrupts are disabled and re-enabled.
- Table mapping each shared structure to its lock (start from the Foundation Guide, verify against source, share with Members 1 and 2 early).
- Exact critical-section boundaries in `kalloc()`, `kfree()` and the scheduler, as function names and statement descriptions (line numbers only from your source).
- Short explanation of why the lock-free version fails in terms of what `acquire()` was guaranteeing.

Oral explanation scaffold

> *A spinlock holds a flag. acquire disables interrupts, then repeatedly does an atomic swap that writes 1 and returns the old value; only the CPU that sees 0 enters the critical section. release clears the flag after a memory fence and re-enables interrupts. Because every access to the free list takes kmem.lock, only one CPU can modify the list at a time.*

Questions to prepare for

- Why must the test-and-set be atomic?
- Why does acquire disable interrupts before spinning?
- What is the purpose of the memory fence?
- What happens if the same CPU acquires a lock it already holds?
- Why not use one global lock for everything?

MEMBER 4 — Foundations, shared-data inventory and quality review

Mission

Build the vocabulary and the inventory of shared data that make the other sections understandable, then compile the report and audit the evidence. This supports the objectives on understanding race conditions and critical sections and identifying shared kernel data structures.

Topics to learn

- Definitions: race condition, critical section, mutual exclusion, atomicity, deadlock.
- A simple non-kernel example: two threads doing `count++` without a lock (load, add, store).
- Why multi-CPU systems and interrupts create the problem.
- Shared kernel structures in xv6-riscv: `kmem`, `proc[]`, `pid_lock`, `wait_lock`, `bcache`, `ftable`, `itable`, `log`, `tickslock`, pipes.
- Why each structure needs mutual exclusion, in one sentence each.

Optional supporting demo (outside XV6, clearly labelled)

In Ubuntu/WSL, write a small C program with two `pthread` threads each doing `counter++` a million times. Without a mutex the final value is usually less than expected; with `pthread_mutex_lock` it is exact. This is a **Linux demonstration of the idea, not an XV6 result**, and must be labelled that way.

```
gcc -O0 -pthread count_race.c -o count_race && ./count_race   # run several times
```

Commands

```
cd ~/xv6-riscv
grep -n 'initlock' kernel/*.c           # find every lock and what it is named
grep -n 'struct spinlock' kernel/*.c    # find structures that embed a lock
```

Your deliverables

- A plain-language glossary of core terms.
- The shared-data inventory table (structure, file, lock, why protected).
- The introduction, abstract and final report formatting; page numbers and slide count checked.
- A limitations paragraph separating observed behaviour from source-code interpretation.
- A final evidence and reference audit: citations, diagram labels, screenshot readability, consistency with the source revision.

Oral explanation scaffold

> *A race condition happens when the result depends on the timing of operations by different CPUs on shared data. The code that touches that data is a critical section, and mutual exclusion means only one CPU runs it at a time. XV6 shares the free list, process table, buffer cache and other structures across CPUs, so each is protected by a lock.*

Questions to prepare for

- Give a non-kernel example of a lost update.
- Name three shared kernel structures and their locks.
- Why are interrupts also a source of concurrency problems?
- What can and cannot be concluded from the experiment?

Integration plan and evidence map

The rubric for your course may differ from the one used by other groups. Confirm the marks breakdown with your faculty and adapt this table. The objectives below come directly from the case study.

Objective / evidence	Evidence to show	Owner
Understand race conditions and critical sections	Definitions, `count++` example, critical-section boundaries	Member 4; Member 3
Identify shared kernel data structures	Inventory table with locks	Member 4; Member 3
Need for mutual exclusion	Interleaving timelines and incorrect-outcome table	Member 2
Effect of concurrent updates	Lock-free test runs, screenshots, results table	Member 1
How locks prevent inconsistent results	Spinlock flow diagram, locked vs lock-free comparison	Member 3; Member 1
Discussion, limitations, conclusion	What was observed, what was not tested	Member 4; all review

Suggested team sequence

- **Meeting 1:** all read the Common Foundation Guide and agree on the repository revision, diagram and vocabulary.
- **Meeting 2:** members research their sections. Member 1 builds and boots XV6 and writes the stress program; Member 3 shares the lock-to-structure table early.
- **Meeting 3:** run the experiment together (baseline, lock removed, lock restored) while Member 2 compares results with the predicted interleavings.
- **Meeting 4:** each member teaches their part to the other three.
- **Final review:** verify sources, screenshot quality, result claims, report page numbers and slide count; confirm `kalloc.c` is restored.

Dependencies

- Member 4's definitions come first; everyone builds on them.
- Member 3's lock map feeds Members 1 and 2.
- Member 1's real results must be shared with Member 2 so theory and observation agree.

Shared rule

This is ownership division, not a division of all knowledge. Every member should be able to explain the free-list race, the spinlock mechanism, the experiment and its limitations. The assigned lead should be the strongest explainer for their section.

References

- xv6-riscv source: https://github.com/mit-pdos/xv6-riscv
- xv6 book (RISC-V edition), Chapter 6: Locking; MIT course materials: https://pdos.csail.mit.edu/6.828/
- Use your checked-out source as the authority for implementation details; list only sources actually consulted.